



3

13

1560 SP

عملي

كلية الهندسة المعلوماتية

السنة الثالثة

Transformation



عمل طلابي

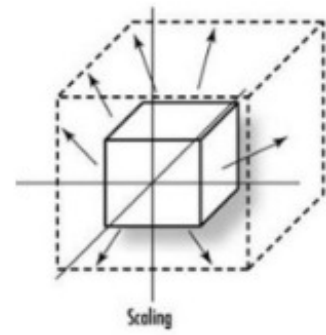
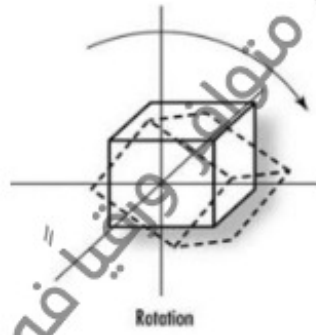
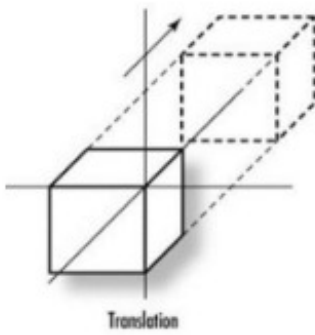
محتوى مجاني غير مخصص للبيع التجاري

3/11/2024

بيانات حاسوبية

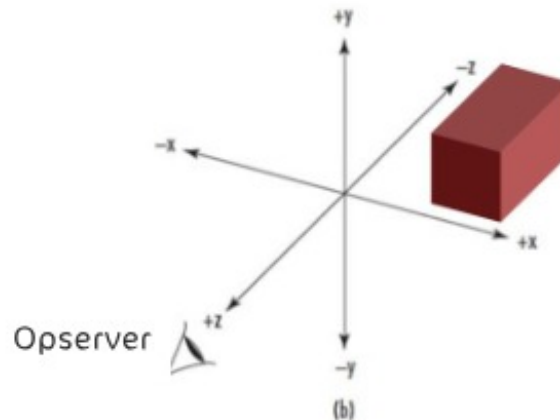
التحويلات في OpenGL (Modeling Transformations)

- تستخدم التحويلات (Modeling Transformations) في تعديل النموذج وتحريك الأجسام الموجودة داخله.
- هذه التحويلات تمكننا من تغيير مواقع الأجسام وتدويرها وتغيير حجمها بما يناسب المشهد.
- الصور أدناه توضح ثلاثة من أكثر التحويلات شيوعاً التي يمكننا تطبيقها على الأجسام:



مركز عالم OpenGL:

في OpenGL، يتكون العالم ثلاثي الأبعاد من نظام إحداثيات (Coordinates System) ثلاثي الأبعاد متمركز حول النقطة $(0, 0, 0)$ ، وهي مركز هذا العالم. يتم تحديد مواقع الأجسام التي يتم رسمها وأحجامها وطريقة تدويرها بناءً على نظام الإحداثيات هذا.



كيف يتم التعامل مع مركز العالم في OpenGL؟

عندما نقوم بإنشاء مشهد في OpenGL، يتم تحديد موضع الأجسام التي نرسمها نسبة إلى مركز العالم الذي يعتبر النقطة المرجعية لكل الأجسام، وعندما نقوم بإجراء أي نوع من التحويلات السابقة (تحريك، تدوير، تغيير حجم)، فإنه يؤثر على جميع الأجسام بشكل نسبي وذلك بناءً على موضعهم نسبة إلى مركز العالم.

Identity matrix (المصفوفة الواحدة)

تعريف المصفوفة الواحدة رياضياً:

هي مصفوفة مربعة (أي لها عدد متساو من الصفوف والأعمدة) وتكون جميع عناصر القطر الرئيسي (العناصر من أعلى اليسار إلى أسفل اليمين) تساوي الواحد بينما بقية العناصر تكون تساوي الصفر. يمكن اختزال عالم OpenGL بالكامل بمصفوفة واحدة أبعادها 4×4 تقابل معلومات مركز الإحداثيات وبالتالي فإن تغيير أي عنصر من هذه المصفوفة يؤدي إلى إحداث تغيير على مركز الإحداثيات، فمثلاً تحوي العناصر في العمود الأخير من الأسطر الثلاثة الأولى على معلومات مركز الإحداثيات والتي في البداية تكون (0, 0, 0).

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{matrix} X \\ Y \\ Z \end{matrix} \rightarrow \text{Center position}$$

وسنلاحظ لاحقاً أهميتها كبدية لعمليات التحويل، ففي الرسومات الحاسوبية تستخدم المصفوفة الواحدة كنقطة بداية أو كـ "وضع افتراضي" للجسم للانطلاق منها للعديد من العمليات التحويلية (مثل التحريك والتدوير وتغيير الحجم)، وتحدث الانتقالات عن طريق ضرب المصفوفة الواحدة بمصفوفة التحويل، لتحديث وضع الجسم تدريجياً في الفضاء ثلاثي البعد. سنرمز لهذه المصفوفة بالرمز M اختصاراً.

التحويلات وتأثيرها على مركز العالم:

فكرة هامة: عندما نقوم بتطبيق أحد التحويلات على جسم في OpenGL كتدويره أو تدويره، فإننا فعلياً لا نقوم بتحريك أو تدوير الجسم بحد ذاته، وإنما نقوم بتحريك أو تدوير نظام الإحداثيات الذي يحتوي على الجسم (أي تحريك عالم OpenGL ثلاثي الأبعاد الذي يحتوي على الجسم وتغيير طريقة عرض الأجسام به)، وبالتالي يظهر الجسم على الشاشة وكأنه قد تحرك.

شرح مبسط للموضوع:

في البداية: يتم رسم جميع الأجسام في مواقعها الافتراضية بالنسبة لمركز العالم (0, 0, 0). عند تطبيق الانسحاب (Translation): يتم نقل نظام الإحداثيات بالكامل بمقدار معين، مما يجعل الجسم يظهر وكأنه قد تحرك.

عند تطبيق الدوران (Rotation): يتم تدوير نظام الإحداثيات، مما يجعل الجسم يظهر وكأنه قد دُور. عند تطبيق التكبير أو التصغير (Scaling): يتم تغيير حجم نظام الإحداثيات، مما يجعل الجسم يبدو وكأنه قد تغير حجمه.

والآن بعد شرح فكرة نظام الإحداثيات سنقوم بالتطرق إلى كل تحويل بالتفصيل:

الانسحاب (Translation) في OpenGL:

الـ Translation هو تطبيق انسحاب على نظام الإحداثيات بمقدار معين على المحاور x, y, z ، أي أننا بمعنى آخر نقوم بنقل مبدأ الإحداثيات من (x, y, z) إلى $(x + dx, y + dy, z + dz)$.

1. تطبيق الـ Translation:

يمكننا تطبيق الـ Translation باستخدام الدالة:

`glTranslatef(dx, dy, dz);`

حيث أن $(x + dx, y + dy, z + dz)$ هي إحداثيات مبدأ الإحداثيات الجديد بفرض أن (x, y, z) هي إحداثيات المبدأ القديم، فمثلاً بعد تطبيق الانسحاب ورسم النقطة `glVertex3f(0, 0, 0)` فإنها سترسم في الإحداثيات $(x + dx, y + dy, z + dz)$ باعتبارها مبدأ الإحداثيات الجديد أي الـ $(0, 0, 0)$ الجديدة.

ملاحظة: يوجد أيضاً التابع `glTranslated(dx, dy, dz)` والذي يختلف عن السابق أن الـ `datatypes` الخاصة بالـ `parameters` هي `double` بدلاً من `float`.

2. مصفوفة الـ Translation:

من الناحية الرياضية، يتم تطبيق جميع التحويلات عن طريق ضرب مصفوفة التحويل الخاصة بها بالمصفوفة M . عند استخدام الدالة `glTranslatef(dx, dy, dz)`، يتم إنشاء مصفوفة `translate` (نرمز لها T)، والتي إذا ضربناها بالمصفوفة M التي ذكرناها سابقاً (المصفوفة الواحدة المعبرة عن معلومات مبدأ الإحداثيات) سينتج المصفوفة MT وسيتم تحريك الأجسام في الفضاء تبعاً لها.

تمثل المصفوفة الـ Translation كالتالي:

$$T = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

🔴 (d_x) : المسافة على المحور x .

🔴 (d_y) : المسافة على المحور y .

🔴 (d_z) : المسافة على المحور z .

التحويل الرياضي باستخدام المصفوفات:

عند العمل مع التحويلات في OpenGL، نبدأ عادةً بالمصفوفة M التي تستخدم كقاعدة لأي تحويل، حيث تمثل الحالة التي لا يوجد فيها أي تحويل.

🔴 الأعداد في القطر الرئيسي (القطر من أعلى اليسار إلى أسفل اليمين) تساوي 1، مما يعني أن القيم لم تتغير بعد.

🔴 الأعداد الأخرى تساوي 0، مما يعني أنه لا يوجد تأثير على الأبعاد الأخرى.

🔴 يتم إعادة تعيين شكل المصفوفة هذا في كل مرة يتم فيها استدعاء التابع `glLoadIdentity()` (أي إعادتها

لـ M).

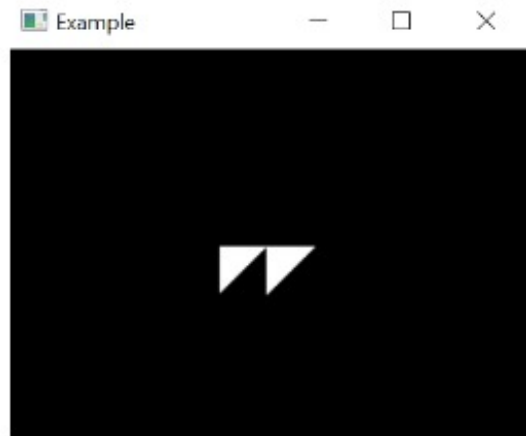
ملاحظات مهمة:

1. في بداية تابع DrawGLScene يجب إعادة المصفوفة لوضعها الأصلي M عن طريق استدعاء التابع `glLoadIdentity()`.
2. عند إجراء أي نوع من التحويلات مثلاً قمنا بانسحاب بمقدار 2 على محور x، فإن جميع الأجسام المرسومة قبل استدعاء تابع `glTranslated` لن تتأثر بهذا الانسحاب وستبقى في مكانها الأصلي، بينما جميع الأجسام التي سترسم بعده هي التي ستتأثر به.
3. غالباً يتم استخدام تابع `glTranslated` في بداية تابع DrawGLScene (بعد استدعاء `glLoadIdentity()`) مع إسناد قيم 0 لكل من x و y وإسناد قيمة سالبة لـ z لأنه كما ذكرنا سابقاً، يجب أن تكون قيم z سالبة ليظهر الشكل المرسوم على الشاشة، ولتجنب كتابة z سالبة في كل مرة نعرف فيها نقطة ما، نقوم بتطبيق انسحاب على المحور z ليصبح مبدأ الإحداثيات بعيداً عن الشاشة بعداً كافياً لظهور الأجسام التي نرسمها دون الحاجة لإسناد قيم z سالبة لها في كل مرة.

3. مثال على الـ Translation:

لنلق نظرة على كيفية استخدام الـ Translation لرسم الأجسام:

```
// إعادة المصفوفة لوضعها الأصلي M
glLoadIdentity();
// تطبيق انسحاب لمركز الإحداثيات على المحور z
glTranslated(0, 0, -10);
// رسم مثلث
glBegin(GL_TRIANGLES);
glVertex2d(0, 0);
glVertex2d(-1, 0);
glVertex2d(-1, -1);
glEnd();
// تطبيق الـ Translation
glTranslated(1, 0, 0);
// نقل النظام بمقدار 1 على المحور x
// رسم مثلث آخر بعد الـ Translation
glBegin(GL_TRIANGLES);
glVertex2d(0, 0);
glVertex2d(-1, 0);
glVertex2d(-1, -1);
glEnd();
```



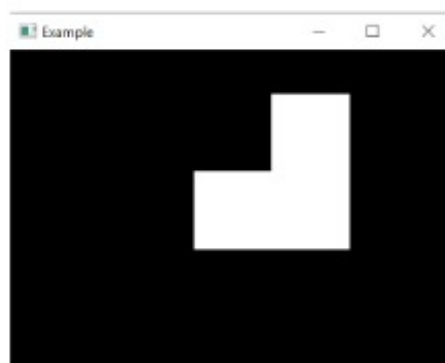
ماذا يحدث هنا؟

يتم رسم مثلث في البداية. بعد ذلك، نقوم بنقل النظام بمقدار 1 على المحور x. عند رسم المثلث الثاني، يبدو وكأنه قد تم نقله إلى اليمين، لكننا لم نغير إحداثياته، بل قمنا بتحريك نظام الإحداثيات.

كيف تعمل المصفوفات عند تطبيق أكثر من تحويل للانسحاب؟

لننظر إلى مثال عملي حيث نقوم بتطبيق عدة تحويلات للانسحاب على شكل ما. سنقوم بتحريك مربع في ثلاث خطوات.

```
glLoadIdentity();
glTranslated(0, 0, -10);
// الخطوة 1: رسم المربع الأول
glBegin(GL_QUADS);
glVertex3d(-1, -1, 0);
glVertex3d(1, -1, 0);
glVertex3d(1, 1, 0);
glVertex3d(-1, 1, 0);
glEnd();
// الخطوة 2: تطبيق الانسحاب الأول
glTranslatef(2.0f, 0.0f, 0.0f); // نقل المربع بمقدار وحدتين على المحور x
// الخطوة 3: رسم المربع الثاني
glBegin(GL_QUADS);
glVertex3d(-1, -1, 0);
glVertex3d(1, -1, 0);
glVertex3d(1, 1, 0);
glVertex3d(-1, 1, 0);
glEnd();
// الخطوة 4: تطبيق الانسحاب الثاني
glTranslatef(0.0f, 2.0f, 0.0f); // نقل المربع بمقدار وحدتين على المحور y
// الخطوة 5: رسم المربع الثالث
glBegin(GL_QUADS);
glVertex3d(-1, -1, 0);
glVertex3d(1, -1, 0);
glVertex3d(1, 1, 0);
glVertex3d(-1, 1, 0);
glEnd();
```



ماذا يحدث في المصفوفة؟

أولاً:

نعيد حالة المصفوفة إلى الحالة الابتدائية M ثم نضربها بمصفوفة التحويل T فتصبح:

$$MT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -10 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

الخطوة 1:

عند رسم المربع الأول، يتم استخدام المصفوفة MT المعدلة، ويظهر المربع في موقعه الافتراضي $(0, 0, 0)$ بالنسبة لمركز الإحداثيات الجديد (ولكنه في الواقع في الإحداثيات الجديدة $(0, 0, -10)$).

الخطوة 2:

نقوم بتطبيق الانسحاب باستخدام $glTranslatef(2.0f, 0.0f, 0.0f)$.
تتحدث المصفوفة MT إلى MTT :

$$MTT = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -10 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

الخطوة 3:

عندما نرسم المربع الثاني بعد تطبيق الانسحاب، يُرسم المربع في الموقع الجديد (بانسحاب وحدتين على x).

الخطوة 4:

عند تطبيق الانسحاب الثاني باستخدام $glTranslatef(0.0f, 2.0f, 0.0f)$ ، فتتحدث المصفوفة الحالية لتصبح:

$$MTTT = \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & -10 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

هنا، يطبق انسحاب بمقدار 2 على المحور y .

الخطوة 5:

عند رسم المربع الثالث، يُرسم في الموقع الجديد فبعد تطبيق كل التحويلات السابقة، سيكون المربع الثالث قد تم سحبه بمقدار 2 إلى اليمين و 2 إلى الأعلى بالنسبة لموقعه الأصلي.

التأثير التراكمي:

مصفوفة التحويل تتراكم كلما قمنا بتطبيق تحويلات إضافية. بمعنى آخر، كل تحويل يعتمد على التحويلات السابقة. عندما نقوم بتطبيق $glTranslatef$ ، فإننا نضيف قيم الإزاحة إلى المصفوفة الحالية، مما يتيح لنا بالتحكم في موضع الأجسام بطريقة متسلسلة.

الدوران (Rotation) في OpenGL:

الدوران هو عملية تغيير توجيه الأجسام حول محور أو نقطة معينة. في OpenGL، يمكن تطبيق هذه العملية بسهولة باستخدام الدالة `glRotatef` أو `glRotated`.

1. تطبيق الـ Rotation:

لتطبيق الدوران على جسم ما في OpenGL، نستخدم الدالة التالية:

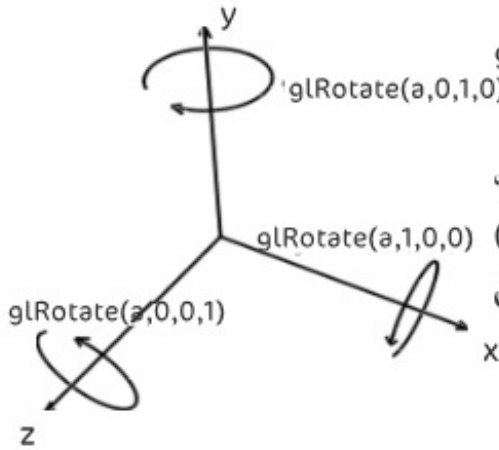
`glRotatef(angle, x, y, z);`

angle: هو مقدار الدوران بالدرجات. يُحدد مقدار التدوير حول المحور المحدد.

x, y, z: هذه القيم تحدد النقطة (x, y, z) التي تساعدنا في تحديد محور الدوران،

بحيث أن التدوير سيتم حول المحور الواصل بين مبدأ الإحداثيات (0, 0, 0) والنقطة التي حددناها (x, y, z)، فمثلاً في حال كانت $x = 0, y = 0, z = 1$ فإن

الدوران سيتم حول المحور z.



2. مصفوفة الـ Rotation:

عند استخدام الدالة `glRotatef(angle, x, y, z)` يتم إنشاء مصفوفة Rotate (نرمز لها R)، والتي إذا ضربناها بالمصفوفة M سينتج المصفوفة MR وسيتم تدوير الأجسام في الفضاء تبعاً لها. هناك ثلاث مصفوفات خاصة للدوران، واحدة لكل محور:

دوران حول المحور x:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

دوران حول المحور y:

$$R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

دوران حول المحور z:

$$R_z(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3. مثال على ال Rotation:

```
glTranslated(0, 0, -10);
// رسم المثلث قبل الدوران
glBegin(GL_TRIANGLES);
glVertex2d(0, 0);
glVertex2d(-1, 0);
glVertex2d(-1, -1);
glEnd();
// تطبيق الدوران بمقدار 90 درجة حول محور z
glRotated(90, 0, 0, 1);
// رسم المثلث بعد الدوران
glBegin(GL_TRIANGLES);
glVertex2d(0, 0);
glVertex2d(-1, 0);
glVertex2d(-1, -1);
glEnd();
```



ماذا يحدث هنا؟

في البداية، يتم رسم مثلث في موضعه الافتراضي. بعد ذلك، يتم تطبيق دوران بمقدار 90 درجة حول المحور z. المثلث الثاني يتم رسمه في الموضع الجديد بعد التدوير، دون الحاجة إلى معرفة الإحداثيات الجديدة له.

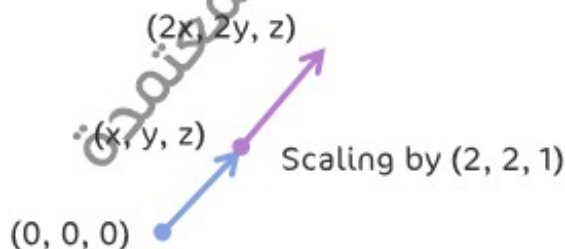
تغيير الحجم (Scaling) في OpenGL:

تغيير الحجم هو عملية تغيير أبعاد الشكل وفقاً لمحاور معينة، مما يسمح لنا بتكبير أو تصغير الأجسام في المشهد ثلاثي الأبعاد.

1. تطبيق ال Scaling:

في OpenGL، يمكننا تطبيق ال scaling باستخدام الدالة التالية:

```
glScalef(xAxis, yAxis, zAxis);
```



xAxis: معامل تغيير الحجم على المحور x.

yAxis: معامل تغيير الحجم على المحور y.

zAxis: معامل تغيير الحجم على المحور z.



2. مصفوفة ال Scaling:

عند استخدام الدالة $glScalef(x, y, z)$ يتم إنشاء مصفوفة Scale (نرمز لها S)، والتي إذا ضربناها بالمصفوفة M سينتج المصفوفة MS وسيتم تغيير حجم الأجسام في الفضاء تبعاً لها.



$$S = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

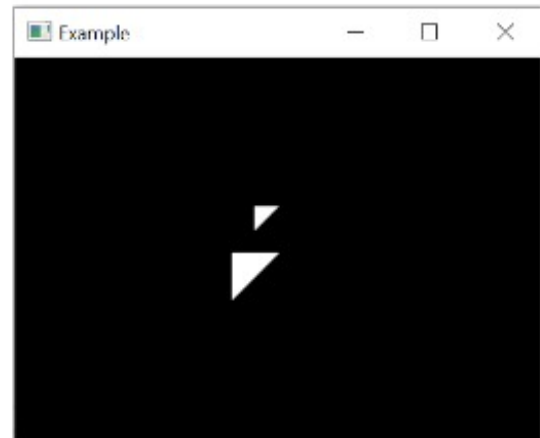
حيث:

 S_x : معامل تغيير الحجم على المحور x . S_y : معامل تغيير الحجم على المحور y . S_z : معامل تغيير الحجم على المحور z .ملاحظة مهمة:

يجب ألا يكون أي من معاملات الدالة صفراً، لأن ذلك سيؤدي إلى اختفاء ذلك المحور. على سبيل المثال، إذا طبقنا $glScalef(0, 1, 1)$ ، فإن المحور x سيختفي بالكامل.

3. مثال على ال Scaling:

```
glTranslated(0, 0, -10);
// رسم المثلث قبل تغيير الحجم
glBegin(GL_TRIANGLES);
glVertex2d(0, 0);
glVertex2d(-1, 0);
glVertex2d(-1, -1);
glEnd();
// تطبيق الانسحاب لنقل الشكل لأعلى
glTranslated(0, 1, 0);
// تطبيق تغيير الحجم على الشكل
glScaled(0.5, 0.5, 0.5); // تصغير الحجم بمقدار 50% على جميع المحاور
// رسم مثلث بعد تغيير الحجم
glBegin(GL_TRIANGLES);
glVertex2d(0, 0);
glVertex2d(-1, 0);
glVertex2d(-1, -1);
glEnd();
```

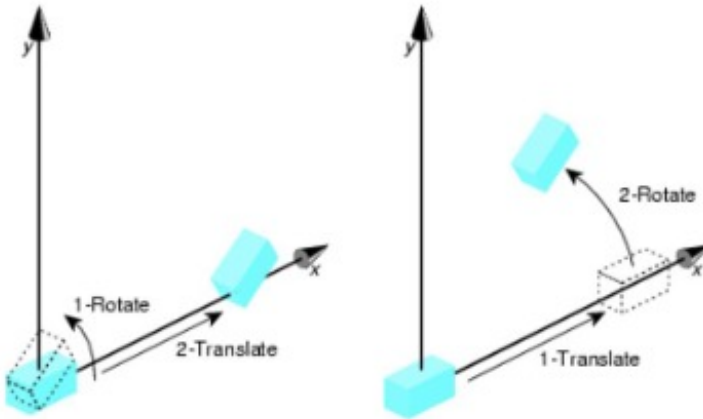


ماذا يحدث هنا؟

في البداية، يتم رسم مثلث بحجمه الافتراضي. بعد ذلك، يتم استخدام `glTranslated (0, 1, 0)` لنقل الشكل لأعلى بمقدار واحد على المحور `y`.

ثم يتم تطبيق `glScaled (0.5, 0.5, 0.5)` مما يؤدي إلى تصغير الحجم بمقدار 50% على جميع المحاور. عند رسم المثلث الثاني، يظهر بحجم أصغر بعد تطبيق تغيير الحجم.

Mixing them all:



بما أن كل تحويل من التحويلات الثلاث السابقة يعادل ضرب المصفوفات وبالتالي فإنها ليست تبديلية، أي أننا إذا قمنا بهذه التحويلات معاً فإن النتيجة ستختلف باختلاف ترتيب هذه التحويلات.

الصورة جانباً توضح أننا لو قمنا بـ `Rotation` على الشكل ثم قمنا بـ `Translation` له، فإنه لا يكافئ قيامنا بالـ `Translation` أولاً ثم بالـ `Rotation`.

التحريك (Animation):

مثال على التحريك:

يمكننا استخدام الـ `Translation` لإنشاء تأثيرات التحريك عبر تحديث موضع الكائن في كل إطار:

```
void Draw_Square() {
    glBegin(GL_QUADS);
    glColor3f(0.4, 0.3, 0.7); // لون المربع
    glVertex3d(-1, -1, 0);
    glVertex3d(-1, 1, 0);
    glVertex3d(1, 1, 0);
    glVertex3d(1, -1, 0);
    glEnd();
}
```

```
// في دالة رسم المشهد
void DrawGLScene() {
    glLoadIdentity();
    glTranslated(0, 0, -10);
    static float X = 0; // إحداثي المتغير X
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glTranslated(X, 0, 0); // نقل المربع
    Draw_Square(); // رسم المربع
    X += 0.001; // زيادة إحداثي X لتحريك المربع تدريجياً
}
```

شرح التحريك:

كما نعلم، فإن التابع DrawGLScene يتم تنفيذه 24 مرة بالثانية، ولذلك نستطيع الاستفادة من هذه الخاصية لمحاكاة الحركة.

بدايةً نقوم برسم مربع يقع مركزه في مبدأ الإحداثيات، ثم يجب أن نقوم بتحريكه بمقدار ضئيل في كل مرة ليظهر وكأنه يتحرك على المحور x، وللقيام بذلك يجب تمرير مقدار الانسحاب dx على شكل متحول يزداد بمقدار ضئيل في كل مرة نستدعي به التابع، ففي المرة الأولى تكون $X = 0$ وفي المرة الثانية تكون $X = 0.001$ وفي المرة الثالثة تكون $X = 0.002$ وتظل تزداد مع كل استدعاء للتابع وبالتالي يتم تطبيق الانسحاب باستمرار وبشكل سريع فيظهر المربع في التنفيذ النهائي وكأنه يتحرك على طول المحور x.

ملاحظة: يمكنكم أيضاً تجربة التحريك باستخدام التحويلات Rotation و Scaling أيضاً وذلك بجعل الزاوية متغيرة أو بجعل مقدار تغيير الحجم متغير.

glPopMatrix و glPushMatrix

كما ذكرنا سابقاً في التحويلات، فإن المصفوفة M يتم ضربها بمصفوفة التحويل في كل مرة نستخدم بها أحد توابع التحويل، فمثلاً عند تطبيق التحويلات:

$loadIdentity \rightarrow Translate \rightarrow rotate \rightarrow scale \rightarrow translate$

ستتغير المصفوفة بهذه الخطوات:

$M \rightarrow MT \rightarrow MTR \rightarrow MTRS \rightarrow MTRST$

ولكن سنواجه مشكلة، ماذا لو كنا نريد العودة لأحد حالات المصفوفة القديمة كالحالة MTR ؟

في هذه الحالة نستطيع استخدام التابع `glPushMatrix()` الذي يقوم بحفظ الحالة الحالية للمصفوفة، وعندما نريد استرجاع آخر نسخة محفوظة نستدعي التابع `glPopMatrix()` فستعود حالة المصفوفة للحالة التي كانت عليها قبل آخر استدعاء لـ `glPushMatrix`.

ملاحظة: يتم تخزين حالات المصفوفة عند استدعاء التابع `push` في `stack`، ولذلك يمكن كتابة `push` و `pop` بشكل متداخل.

فمثلاً عند تطبيق هذه التحويلات:

$loadIdentity \rightarrow translate \rightarrow \text{push} \rightarrow rotate \rightarrow \text{push} \rightarrow translate \rightarrow \text{pop} \rightarrow scale \rightarrow \text{pop}$

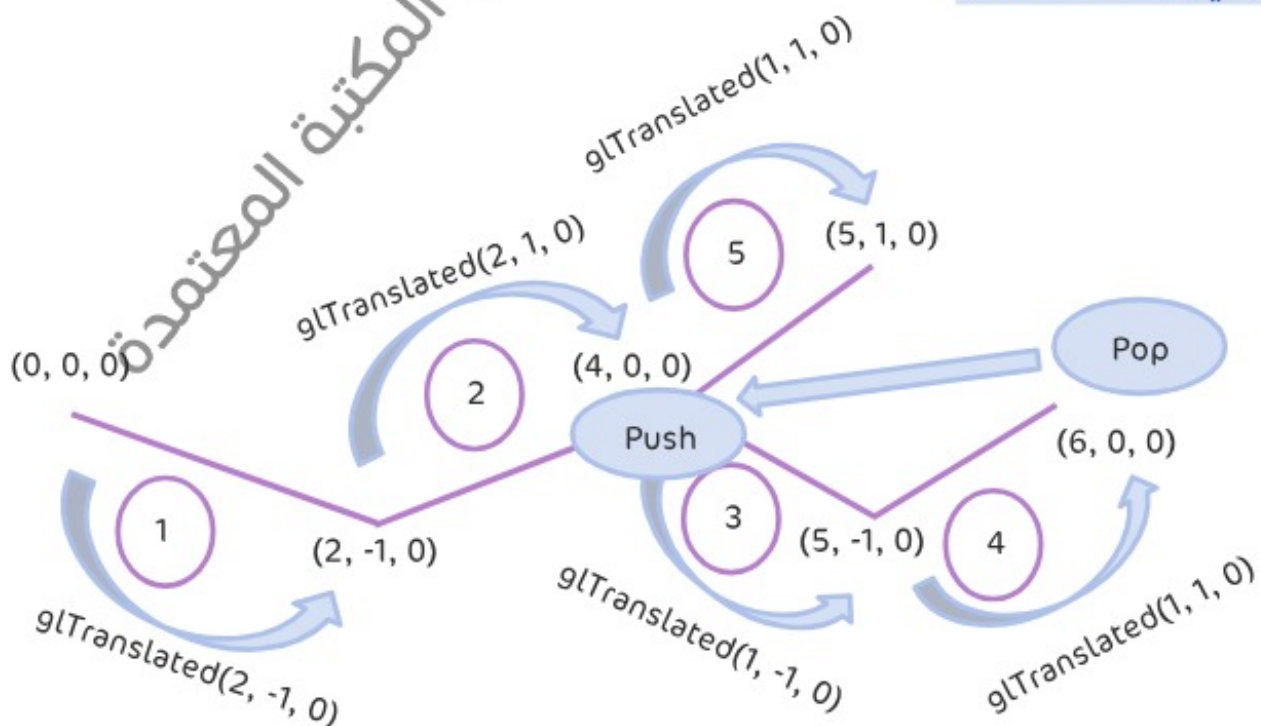
سيتم تطبيق هذه التحويلات على المصفوفة M :

$M \rightarrow MT \rightarrow \text{save MT in the stack} \rightarrow MTR \rightarrow \text{save MTR in the stack} \rightarrow MTRT$
 $\rightarrow \text{Retrieve the last matrix stored in the stack (MTR) and remove it} \rightarrow MTRS$
 $\rightarrow \text{Retrieve the last matrix stored in the stack (MT) and remove it.}$

مثال توضيحي:

```
glPushMatrix(); // حفظ الحالة الحالية
glTranslatef(1.0f, 1.0f, 0.0f); // تطبيق الانسحاب
Draw_Square(); // رسم الشكل
glPopMatrix(); // استرجاع الحالة السابقة
```

مثال رسومي لإيصال الهدف أكثر:



Depth Buffer

مقدمة: الـ Depth Buffer (أو Z-buffer) يستخدم لتحديد العمق (الإحداثيات z) لكل بكسل في الرسومات الثلاثية الأبعاد. حيث أنه عندما يتم رسم كائنات على الشاشة، يتم إسناد قيمة z لكل بكسل، والتي تمثل المسافة بين الكائن وعين المراقب.

كيف يعمل الـ Depth Buffer:

- أولاً يتم رسم البكسل الأول مع قيمة z معينة.
- عند رسم بكسل آخر على نفس الموضع، يتم فحص قيمة z الجديدة مقابل القيمة المرسومة سابقاً.
- إذا كانت القيمة الجديدة أصغر (أقرب)، يتم رسم النقطة الجديدة وتحديث قيمة z.
- إذا كانت القيمة الجديدة أكبر (أبعد)، يتم تجاهل الرسم.

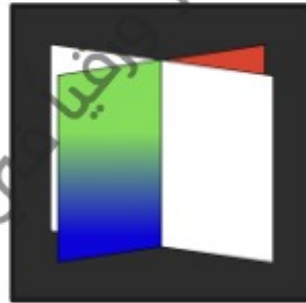
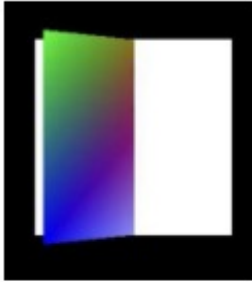
في حال تفعيل Depth Buffer:

يحصل التقاطع بين الشكلين مما يجسد واقعية وفهم أكثر للمشاهد.

لتفعيل Depth Buffer نكتب هذه التعليمة في التابع InitGL:

```
glEnable(GL_DEPTH_TEST);
```

في الشكل السابق يوجد مربع أبيض ومربع ملون متداخل معه، ولأن الـ Depth Buffer مفعل، لم يظهر الجزء الخلفي من المربع الملون.

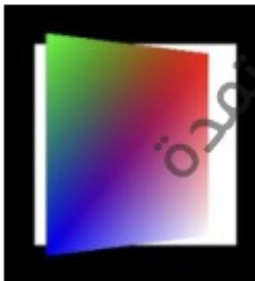


في حال إلغاء تفعيل Depth Buffer:

لا يحصل التقاطع مما يسبب ارتباك في فهم المشهد.

لإلغاء تفعيله نكتب هذه التعليمة في التابع InitGL:

```
glDisable(GL_DEPTH_TEST);
```



“you only live once, but if you
work it right,
once is enough”

